

Stage 2: Data Preprocessing & Context-Aware Chunking

Multimodal Extraction, Audit Storage & RAG-Ready Chunk Generation

1. Executive Goal of Stage 2

In Stage 1, we gathered approximately **1.04 GB of raw assets** in **data/raw/**, including unrendered HTML text, multi-page PDFs, Word document schedules, Google Sheets, and OCR-extracted recruiter text.

Raw files cannot be directly fed into vector databases or LLMs because:

- **Structural noise:** script tags, styling, layout tables, and other presentation artifacts.
- **Context-window constraints:** documents may exceed embedding-model and LLM context limits.
- **Information loss:** blind slicing can separate values from their department, course, or program headers.

Stage 2 converts raw binary files into clean standardized text in **data/processed/extracted_text/** and then creates metadata-enriched passages in **data/processed/chunks.json**.

2. Technical Workflow Architecture

```
[ data/raw/ ] (1.04 GB Raw Assets)
■
■■■■ HTML Pages ■■■■ Plain-Text Cleaning
■■■■ PDFs ■■■■ pdfplumber (Body Text + Grid Tables)
■■■■ DOCX ■■■■ python-docx (Paragraphs + Schedule Tables)
■■■■ CSV / XLSX ■■■■ pandas (Key-Value Row Formatting)
■■■■ Images ■■■■ Tesseract OCR Text
■
▼
[ data/processed/extracted_text/ ]
■
▼
[ Context-Aware Chunker Engine ]
512-word sliding window • 60-word overlap
Metadata: title + category + source URL
■
▼
[ data/processed/chunks.json ]
■
▼
[ FAISS Dense Index + BM25 Keyword Index ]
```

3. Step-by-Step Breakdown of What We Implemented

Step 1: Multimodal File Extraction (extract_text_*)

- **PDF Parser (pdfplumber):** Extracts body text page-by-page and embedded grid tables, formatting tables as Markdown-style pipe-separated strings such as **col1 | col2 | col3**.
- **Word Document Parser (python-docx):** Iterates through paragraphs and table-cell matrices to capture class timetables and lab schedules without dropping rows.
- **Spreadsheet Parser (pandas):** Converts CSV/XLSX rows into key-value strings, e.g. **Row 1: Course: Data Structures | Code: CSL202 | Slot: A**.
- **Recruiter Image Integration:** Merges Tesseract OCR-extracted text from company-logo grids into the relevant placement document text.

Step 2: Dual Storage & Audit Logging (extracted_text/)

Before chunking, the pipeline writes a physical **.txt** file for every cataloged asset into **data/processed/extracted_text/**, named using its **doc_id**.

Why this matters: It provides a human-readable audit repository proving that the multimodal assets were successfully parsed, while also making debugging and faculty evaluation easier.

Step 3: Context-Aware Sliding Window Chunking (chunk_text_with_metadata)

Instead of static text splitting, the pipeline uses a **metadata-prepended sliding-window strategy**.

- **Chunk parameters:** approximately **512 words** per chunk with a **60-word overlap** to reduce boundary-related context loss.
- **Metadata prepending:** every chunk starts with document lineage information.

Chunk Text = "Document: [Title] | Category: [Category] | Source: [URL]\n\n" + Body Text

Benefit: When retrieval returns a single chunk, the downstream LLM still knows the originating document, category, and source page, reducing context fragmentation.

4. Summary Table for Presentation & Defense

Feature	Engineering Challenge	Our Implementation	Result / Benefit
PDF Extraction	Loss of table context in notices	pdfplumber cell-grid parsing	Preserves column structure for fee tables
Schedule DOCX	Unstructured schedule tables	python-docx table-cell iteration	Captures lab and class timings accurately
Spreadsheets	Raw numbers lose header meaning	pandas row-level key-value mapping	Converts matrix cells into readable sentences
Audit Trail	Proving parsing succeeded	Physical .txt files in extracted_text/	Evidence for evaluation and debugging
Chunk Lineage	Chunks lose document origin	Metadata header prepending	Reduces context fragmentation during RAG retrieval

5. Output Deliverables

- **extracted_text/*.txt:** Complete, human-readable plain-text extractions for every cataloged raw file.
- **chunks.json:** Clean, metadata-enriched chunk payload ready for the **FAISS dense vector index** and **BM25 keyword index** in Stage 3.

Stage 2 outcome: Raw multimodal institutional data is transformed into auditable, standardized, context-preserving text chunks ready for hybrid retrieval in the next RAG stage.